

Basic Data Manipulation in R

BIOL 3401 Mount Royal University

Topic 3, Winter 2026

Contents

Review	1
Introduction to basic coding	3
1 - Basic operations	3
2 - Data structures	5
3 - Working with data structures in R	7

Review

NOTE: All of the code chunks in the review section have “eval = F” in the heading of the code chunk. This means R will not evaluate (run) any of the code within the code chunk. The reason is that the code chunks load data or point to folders that don’t actually exist, which will cause errors. As you “fix” a code chunk, you will have to manually remove the “eval = F” to make the code run.

Setting a working directory

You can use the `setwd()` function to set your directory to the folder containing your files (note: replace the file path within the parentheses below with your own file path!):

```
setwd("Users/user....")
```

Your current working directory appears in the top right corner of your console.

You can also get R to tell you the path to your current working directory using the code:

```
getwd()
```

```
## [1] "/Users/levi/Documents/R-Projects/tutorial_intro_R"
```

Once you have set a working directory, you can use the `read.csv()` function to load the data from within your working directory. Remember to change “data” to the name of the file. You will also need to assign the dataset a name (“data”) using “<-”, as shown below, or R will only print it out in your console. We will talk more about this in lesson 3.

```
data <- read.csv('.../data.csv')
```

If you end up setting your working directory to the wrong folder, or you would like to change directories, you can always move up to your root directory (one folder at a time) with `..` in the `setwd()` function:

```
setwd('..')
```

Or, without setting the working directly, you can copy the full file path into the `read.csv()` function (replace the file path with your local file path):

```
data <- read.csv('Users/user .../data.csv')
```

Mini Challenge 1: Load external data

1. Download the *dinosaurs.csv* file from the Week 1 content on D2L and save it somewhere on your computer.
2. Load the *dinosaurs.csv* dataset by copying the filepath.
3. Set your working directory to the location of the *dinosaurs.csv* dataset then load the dataset from there, calling the dataset “dino_data”.

Use this code block to code your answers!

```
#-----
```

Saving your data

Equally important is knowing how to save your dataset when you want to move it **out** of R or between scripts after working with it. There are two important functions for doing so: `write.csv()` and `saveRDS()`. The `write.csv()` function saves the data as a comma-separated file. The `saveRDS()` function saves it as an “R data serialization” file, which is a file type native to R. This file type is useful for moving datasets between scripts because it is much **smaller** than a comma-separated file, and because it **preserves the data types in your dataset so R can recognize them**.

```
write.csv(data, 'data_updated.csv')
saveRDS(data, 'data_updated.rds')
```

See how each function first takes the name of the dataset, followed by a comma, then the name (and optionally, folder path) you want to use for the saved file. Note that the file will automatically save to the location of your current working directory. If you would like it to save to a different location, you will need to add a path before the file name. Or, if you have already set your working directory, it will save there.

```
saveRDS(data, 'Users/user .../NewDirectoryLocation/data_updated.rds')
```

Remember that file paths are relative to your current working directory. If you have already set a working directory, specifying a full file path for saving *might* return an error because you are essentially asking R to look for that file path starting from within the current working directory.

After saving as an RDS file, you will need the `readRDS()` function to load it again.

```
updated_data <- readRDS('data_updated.rds')
```

Mini Challenge 2: Save the dinosaurs data to a different location

1. On your computer, within your current working directory, create a new folder called “data”.
2. Save the *dinosaurs* dataset to the new “data” folder as an Rds file.

Use this code block to code your answers!

```
#-----
```

You should practice on your own to get comfortable loading and saving data using different approaches!

Introduction to basic coding

1 - Basic operations

We looked at the five basic data types in lecture. Four of them are below.

```
# A character  
"A"
```

```
## [1] "A"
```

```
# A numeric  
1.77
```

```
## [1] 1.77
```

```
# An integer  
2
```

```
## [1] 2
```

```
# A logical  
TRUE
```

```
## [1] TRUE
```

As we saw in lecture, we can perform arithmetic operations on numeric types (including integers).

```
# 2.5  
5 / 2
```

```
## [1] 2.5
```

```
# 10
5 * 2
```

```
## [1] 10
```

```
# 5
2.5 + 2.5
```

```
## [1] 5
```

We can also assign the outputs of these operations to new objects using the assignment operator. This allows us to store the outputs in our environment, then perform operations on them by “calling” their names.

```
# Create x and y objects from operations
x <- 5 * 2
y <- 2.5 + 2.5

# Perform operation on the x and y objects
x - y
```

```
## [1] 5
```

An important thing to note here is that while `x - y` returns 5 as the output, the values of `x` and `y` themselves *do not change* when we perform these operations. That’s because we didn’t store the output as a new object in our environment.

If we want to store the output as a new object, we need to assign a name to the output.

```
# z is the difference between x and y
z <- x - y
```

An important note about best practices (mind the Golems!). You can imagine how easy it would be to overwrite a variable you’ve been updating and performing functions on. *The last thing you want to do is overwrite all your work by accident!* This negates one of the most important perks of using R. Even though you are now saving your code in a script or R Markdown file, some functions on large data sets can take a while to run, and having to re-run everything might take an unacceptable amount of time.

To avoid overwriting your work, always create new, descriptively named objects as you apply functions to your data.

Mini Challenge 3: Turn an object into a factor data type

`read.csv()` and `setwd()` are special operators in R called **functions**. All functions have the same format: the name of the function, followed by parentheses or brackets, “()”. I.e., “function()”. There is a function called “factor” that turns any object into a factor, our fifth data type. Turn the object `x` (a numeric) into a factor, and assign a new name, “`x_factor`”.

Use this code block to code your answers!

```
#_____
```

2 - Data structures

Vectors and lists

Let's start by making one of the simplest R objects, a *vector* of numbers.

```
vec1 <- c(1, 2, 3, 4, 5)
```

`vec1` is an *object* that we created by using the `<-` operator.

`vec1` now contains the output of the *function* `c(1, 2, 3, 4, 5)`, where the `c()` function (c, for **c**ombine) combines “*elements*” (1, 2, 3, 4, 5) into a vector (we will look at more complex ways to combine data later).

When you create an object, you'll notice that your environment pane is no longer empty. Just like your dataset showed up when you loaded data last week, your object `vec1` is now listed as a value in the environment. Here is some information about what this vector contains:

- “num” to state that it is numerical data
- “[1:5]” to show that there are five elements in the vector
- the contents of the vector (1 2 3 4 5)

If we made a vector too large for the environment pane, it would cut off some of the elements in the vector. We could view the whole vector in the console pane using `print()`

```
print(vec1)
```

```
## [1] 1 2 3 4 5
```

or you can type the name of the object in the console (or run it from a script) to display the object.

```
vec1
```

```
## [1] 1 2 3 4 5
```

We can also use the `class` function to get the data type.

```
class(vec1)
```

```
## [1] "numeric"
```

Let's make a few more vectors:

```
# Creates a factor with two "levels", red and orange
c_factor <- factor(c("red", "red", "orange", "orange"))
```

```
# A character vector
letters <- c("a", "b", "c", "d")
```

```
# A logical vector
TF <- c(TRUE, TRUE, FALSE, TRUE, TRUE)
```

With the vectors `letters`, and `c_factor`, we had to use `"` around each letter/word to specify that we want to create a *character* vector.

Mini Challenge 4: Force an error!

1. What message do you get from R when you run this code chunk below that doesn't have "" around the letters? (remember to remove `eval = FALSE`!)

```
letters <- c(a, b, c, d)
```

2. What do you think the message means? Or rather, why do you think you received that message?

Write your answers below!

#-----

We know that *vectors* need to be made up of all the same data type (e.g., every element is character or numeric). The `letters` and `TF` vectors are two different data types (what are they?).

Mini Challenge 5: Combine two different data types

1. Use one of the approaches we've already talked about (either a function or looking in your environment) to confirm the data types of `letters` and `TF`.
2. Figure out an approach to store the `letters` and `TF` vectors together in a single object. (hint: there is a function called *"list"*)

Use this code block to code your answers!

#-----

Data frames

Tabular data is a collection of vectors. The vectors can be (but don't have to be) different types. However, *all have to have the same length*. An example of tabular data is an Excel sheet. An Excel sheet has one or more columns with rows representing separate observations. In R, we'll mostly focus on data frames and tibbles (the tidyverse version of a data frame—more on this next time).

You will be loading your spreadsheets into R to explore, create figures, and analyze the data.

Let's load a data set to play with in R so we can understand how data frames look in R. In our first topic, we loaded the "dinosaurs.csv" dataset as a comma-separated values file. We can load a comma-separated values file using R's basic functionality because it recognizes this file format. However, many people share Excel (xlsx) files, which R doesn't know how to read. One reason is that Excel files can accommodate multiple sheets: one for the dataset, and the other for the metadata. To read Excel files, we need to install a new package called `readxl`.

```
# Install the package!
```

```
# Attach the package  
library(readxl)
```

We are going to load an Excel data file containing counts of sheep parasites. **Make sure you change the file path to the location of the file when you downloaded it or set a working directory.**

The file includes a second sheet with metadata. By default, `read_excel()` only reads in *one* sheet of the spreadsheet. What if we have two sheets (remember to remove `eval == FALSE`)?

```
sheep_parasites <- read_excel("datasets/sheep_parasites.xlsx")
```

... that's not what we want! We need to change the *arguments* in the `read_excel()` function to tell R which sheet to load.

Mini Challenge 6: Getting help

1. Look up the `read_excel()` function in the Help pane (or use `?read_excel()`) to figure out which argument determines the sheet you load.
2. Try loading the `sheep_parasites.xlsx` file tabular data, but not the metadata.

Use this code block to code your answers!

```
#-----
```

We might encounter different file formats for tabular data, e.g., tab-separated values (tsv) datasets. In comma-separated values (csv) files, each row contains an observation and columns are separated by commas. In tsv files, they are separated by tabs spacing. In reality, both file types are simple text-based files that are easy for computers to understand.

We can load both tsvs and csvs (and other 'text-based' files) with `read.table()`.

```
dinosaurs <- read.table("datasets/dinosaurs.tsv")
```

3 - Working with data structures in R

Arithmetic operations on more complex data structures

In the previous section, we performed arithmetic operations on simple objects with a single element. R can also handle *vector math* for us without any extra instructions on our part!

```
vec1
```

```
## [1] 1 2 3 4 5
```

```
vec1 * x
```

```
## [1] 10 20 30 40 50
```

```
vec2 <- vec1 * x  
vec2
```

```
## [1] 10 20 30 40 50
```

Mini Challenge 7: Vector operations

1. Create a new vector called `nums` that contains any 5 numbers you'd like.
2. Create a new vector called `squares` that contains the elements of `nums` squared (hint: $2^2 = 2$ squared).

Use this code block to code your answers!

```
#-----
```

Subsetting using position and relational operators

One of the main reasons you might want to index data is to remove elements you don't want.

We can index the `letters` vector we created earlier. Let's say we want to know which letter is in the first position of the vector.

```
# Indexing the first position of the letters vector
letters[1]
```

```
## [1] "a"
```

We find the first element in the vector is “a”.

Mini Challenge 8: Subsetting vectors

1. Implement *two different approaches* for removing the letter “a” from the `letters` vector.
2. Assign each shortened vector a new name.

Use this code block to code your answers!

```
#-----
```

We can also index data frames using positions.

```
# The second row of the sheep_parasites data frame
sheep_parasites[2,]
```

```
## # A tibble: 1 x 7
##   AgeCat Sex   Northing Easting N_samples_parasites avg_Strongyles_per_s~1 Year
##   <chr> <chr> <chr>    <chr>          <dbl>          <dbl> <dbl>
## 1 Lamb   F     South    East              1             800  1994
## # i abbreviated name: 1: avg_Strongyles_per_sample
```

```
# The third entry in the fourth row of the sheep_parasites data frame
sheep_parasites[4,][3]
```

```
## # A tibble: 1 x 1
##   Northing
##   <chr>
## 1 North
```

If we assigned a name to these indices, we would get a new data frame with only the row or element we want. We can also use relational operators to subset the things we need. For example, let's imagine we want only the adult sheep from the `sheep_parasites` data frame.

```
sheep_adults <- sheep_parasites[sheep_parasites$AgeCat == 'Adult' ,]
```

For columns with numeric data, another useful trick is to subset all values more than, less than, equal to or less than, etc. a particular value within a column. The operator “==” works to subset values *matching* a specific number.

Mini Challenge 9: Subsetting data frames

1. Try to subset all years in the `sheep_parasites` data frame *during* and later than 2013 (hint: “ $x > y$ ” means more than y , and $x < y$ means x less than y).
2. Assign the resulting data frame with any name you’d like.

Use this code block to code your answers!

```
#-----
```

We can also set multiple criteria for subsets using the “&” symbol. For example, if we wanted all female (F) adult sheep, we could use the following code.

```
female_adults <- sheep_parasites[sheep_parasites$AgeCat == "adult" & sheep_parasites$Sex == "F" ,]
```

Combining data

We just subset our data frames and vectors by removing the elements we didn’t want. Equally important is knowing how to *combine* data. We already practiced combining data when we first talked about data structures. We combined elements into vectors using the `c()` function, and we also made a list by combining vectors with two different data types.

We can also bind rows and columns to data frames, or bind two data frames together using the `cbind()` and `rbind()` functions. `cbind()` binds the data frames into a wider format (by column), and `rbind()` binds them into a longer format (by row). We just need to make sure (1) the rows and column numbers match, and (2) if we are binding the data frames by row, the names of the columns must match.

```
# Creating a new data frame made of two vectors with different data types
df1 <- data.frame(col1 = c("A", "B", "C"),
                  col2 = c(1, 2, 3))

# Creating another new data frame with the same number of rows and columns
df2 <- data.frame(col1 = c("D", "E", "F"),
                  col2 = c(4, 5, 6))

# Bind two data frames by column (width-wise)
cbind(df1, df2)
```

```
##   col1 col2 col1 col2
## 1    A    1    D    4
## 2    B    2    E    5
## 3    C    3    F    6
```

Mini Challenge 10: Mismatched data frames

What do you think would happen if you tried to bind two data frames with a different number of rows/columns?

Write your answers below!

```
#-----
```

Summarizing data

Another useful task is summarizing. We can make summaries of the highlights of data frames and vectors to scope our data.

```
# summarizes the moments and quartiles of numeric data and counts each character type  
summary(sheep_parasites)
```

```
##      AgeCat      Sex      Northing      Easting  
## Length:3708 Length:3708 Length:3708 Length:3708  
## Class :character Class :character Class :character Class :character  
## Mode :character Mode :character Mode :character Mode :character  
##  
##  
##  
## N_samples_parasites avg_Strongyles_per_sample      Year  
## Min. :0.000 Min. : 0.0 Min. :1988  
## 1st Qu.:1.000 1st Qu.: 100.0 1st Qu.:1996  
## Median :1.000 Median : 300.0 Median :2004  
## Mean :1.087 Mean : 473.7 Mean :2004  
## 3rd Qu.:1.000 3rd Qu.: 600.0 3rd Qu.:2010  
## Max. :8.000 Max. :4800.0 Max. :2017
```

```
# provides the data type and first several values of each column  
str(sheep_parasites)
```

```
## tibble [3,708 x 7] (S3: tbl_df/tbl/data.frame)  
## $ AgeCat      : chr [1:3708] "Adult" "Lamb" "Yearling" "Lamb" ...  
## $ Sex         : chr [1:3708] "F" "F" "F" "M" ...  
## $ Northing    : chr [1:3708] "South" "South" "South" "North" ...  
## $ Easting     : chr [1:3708] "East" "East" "East" "West" ...  
## $ N_samples_parasites : num [1:3708] 2 1 1 1 2 0 0 1 1 0 ...  
## $ avg_Strongyles_per_sample: num [1:3708] 167 800 500 300 125 0 0 300 700 0 ...  
## $ Year        : num [1:3708] 1997 1994 1995 2010 1994 ...
```

```
# First six elements in the data frame (works for vectors too!)  
head(sheep_parasites)
```

```
## # A tibble: 6 x 7  
##   AgeCat Sex Northing Easting N_samples_parasites avg_Strongyles_per_s~1 Year  
##   <chr> <chr> <chr> <chr> <dbl> <dbl> <dbl>  
## 1 Adult F South East 2 167 1997  
## 2 Lamb F South East 1 800 1994  
## 3 Yearl~ F South East 1 500 1995  
## 4 Lamb M North West 1 300 2010  
## 5 Adult F South East 2 125 1994  
## 6 Lamb F South East 0 0 1992  
## # i abbreviated name: 1: avg_Strongyles_per_sample
```

```
# Last six elements in the data frame (works for vectors too!)  
tail(sheep_parasites)
```

```
## # A tibble: 6 x 7
##   AgeCat Sex   Northing Easting N_samples_parasites avg_Strongyles_per_s~1 Year
##   <chr>  <chr> <chr>    <chr>          <dbl>          <dbl> <dbl>
## 1 Adult  M     North   West             2             200  1995
## 2 Yearl~ M     South   West             1             700  1992
## 3 Lamb   F     South   East             1            3000  2011
## 4 Adult  M     North   East             1             800  2016
## 5 Lamb   M     North   West             1             600  2014
## 6 Yearl~ M     North   East             1            1600  2015
## # i abbreviated name: 1: avg_Strongyles_per_sample
```

We can also get the means of rows and columns in a data frame. We just need to remember to include only numeric data!

We will also want to use some sort of summarizing function along with rowMeans in this case. This is because our sheep_parasites data has almost 4,000 rows, which will give us almost 4,000 means!

```
# Mean values in each column
colMeans(sheep_parasites[, 5:7])
```

```
##      N_samples_parasites avg_Strongyles_per_sample      Year
##      1.087379          473.685275      2003.590615
```

```
# Mean values across each row
head(rowMeans(sheep_parasites[, 5:7]))
```

```
## [1] 722.0000 931.6667 832.0000 770.3333 707.0000 664.0000
```